

SRI KRISHNA COLLEGE OF ENGINEERING AND TECHNOLOGY

An Autonomous Institution | Approved by AICTE | Affiliated to Anna University | Accredited by NAAC with A++ Grade
Kuniamuthur, Coimbatore – 641008
Phone: (0422)-2678001 (7 Lines) | Email: info@skcet.ac.in | Website: www.skcet.ac.in

**SMART BUY ADVISOR v2.0: A REAL-TIME FINANCIAL DECISION ENGINE AND
INVESTMENT OPPORTUNITY COST SIMULATOR FOR OPTIMAL RETAIL
PURCHASING STRATEGIES**

A MINI PROJECT REPORT

Submitted by

NITIN V TIMOTHY (2403727710621136 / 727724EUEC136)

In partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

IN

ELECTRONICS AND COMMUNICATION ENGINEERING

SRI KRISHNA COLLEGE OF ENGINEERING AND TECHNOLOGY

An Autonomous Institution | Approved by AICTE | Affiliated to Anna University | Accredited by NAAC with A++ Grade
Kuniamuthur, Coimbatore – 641008.

OCTOBER 2026

SUSTAINABLE DEVELOPMENT GOALS (SDG)

The Sustainable Development Goals are a collection of 17 global goals designed as a blueprint to achieve a better and more sustainable future for all. Set in 2015 by the United Nations General Assembly and intended to be achieved by the year 2030, 195 nations agreed upon these goals to transform global economic and environmental stability. This project directly addresses key UN SDGs related to financial literacy, responsible consumption, and innovative digital infrastructure.

Question	Answer
Which SDGs does the project directly address?	SDG 8 (Decent Work and Economic Growth), SDG 9 (Industry, Innovation and Infrastructure), and SDG 12 (Responsible Consumption and Production).
What strategies or actions are being implemented to achieve these goals?	By providing an open-access, offline-first digital financial calculator that models retail financing options (Full Cash vs. Standard/No-Cost EMI) and opportunity cost compounding. It prevents predatory debt traps, promotes personal capital accumulation, and optimizes consumer purchasing efficiency through modern web technologies.
How is progress measured and reported in relation to the SDGs?	Through user adoption metrics, net financial capital saved per purchase scenario, reduction in unnecessary interest overhead, user scenario persistence rates, and financial literacy engagement scores.
How were these goals identified as relevant to the project's objectives?	High-value consumer purchases (electronics, vehicles, appliances) heavily impact personal financial stability. Misunderstanding EMI interest structures and opportunity costs leads to capital depletion. The project promotes economic empowerment, financial transparency, and intelligent resource allocation.
Are there any partnerships or collaborations in place to enhance this impact?	The platform can integrate with personal finance management (PFM) APIs, banking institutions, retail e-commerce ecosystems, financial literacy NGOs, and fintech analytics platforms to enhance consumer decision-making.

BONAFIDE CERTIFICATE

Certified that this mini project report titled “**SMART BUY ADVISOR v2.0: A REAL-TIME FINANCIAL DECISION ENGINE AND INVESTMENT OPPORTUNITY COST SIMULATOR FOR OPTIMAL RETAIL PURCHASING STRATEGIES**” is the bonafide work of **NITIN V TIMOTHY (2403727710621136 / 727724EUEC136)** who carried out the mini project under my supervision.

SIGNATURE

SIGNATURE

DR. B. VIJAYALAKSHMI

SUPERVISOR

Assistant Professor

Department of Electronics and

Communication Engineering

Sri Krishna College of Engineering

and Technology

Kuniamuthur, Coimbatore – 641008.

DR. D. MOHANA GEETHA

HEAD OF THE DEPARTMENT

Professor and Head

Department of Electronics and

Communication Engineering

Sri Krishna College of Engineering

and Technology

Kuniamuthur, Coimbatore – 641008.

Submitted for the Project viva-voce examination held on _____

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

At this juncture, I take the opportunity to convey my sincere thanks and heartfelt gratitude to the management of **Sri Krishna College of Engineering and Technology** for providing state-of-the-art infrastructural facilities and computing resources to carry out this project successfully.

I wish to convey my deepest gratitude to my respected Principal, **Dr. K. Porkumaran**, for his continuous encouragement, visionary leadership, and institutional support throughout the academic curriculum.

I express my profound thanks and sincere gratitude to **Dr. D. Mohana Geetha**, Professor and Head of the Department, Department of Electronics and Communication Engineering, for her invaluable guidance, constant motivation, and academic oversight.

I extend my sincere and wholehearted gratitude to my project supervisor, **Dr. B. Vijayalakshmi**, Assistant Professor, Department of Electronics and Communication Engineering, for her constructive critique, technical guidance, and patient mentoring at every stage of the system's design and implementation.

Finally, I express my heartfelt appreciation to my faculty members, laboratory staff, family, and peers who supported me directly and indirectly in transforming this project into a fully functional reality.

ABSTRACT

In modern consumer retail finance, individuals frequently evaluate purchasing decisions purely on immediate cash outlay rather than considering the **opportunity cost of capital**. Consumers possessing sufficient cash to buy high-value items upfront (e.g., smartphones, laptops, electric vehicles) often default to full cash payment under the assumption of avoiding debt. In doing so, they completely forfeit compound investment returns that their unspent principal capital could generate over time. Conversely, consumers opting for EMIs frequently fail to calculate processing fees, bank interest subventions, and the compounding advantage of servicing EMIs via regular salary income while retaining principal savings intact.

To solve this fundamental financial decision bottleneck, this project presents **Smart Buy Advisor v2.0**, a self-contained, client-side reactive financial decision engine built on the **React 18, JavaScript ES6, and Web Storage API** technology stack. The platform consolidates complex retail financing math into an intuitive single-page web application architecture:

1. **Dual Scheme Financing Engine:** Supports real-time calculation of both **No-Cost EMI** (incorporating bank subvention interest discounts) and **Standard EMI** (with customizable annual interest rates and processing fees).
2. **Salary EMI Pledge Simulator (v2.0):** Implements a variable Salary Pledge engine ($S \in [0, 100]\%$) that models monthly EMI servicing from incoming salary. Servicing 100% of EMI from salary leaves initial savings capital 100% untouched, compounding at full annual ROI without monthly withdrawals.
3. **Real Effective Out-of-Pocket Cost Architecture:** Refines result depiction by calculating the **Real Effective Cost of the Item** (Original Purchase Outflow—Investment Returns Earned). This models investment interest as a direct price subvention offset, proving how opportunity returns lower the real out-of-pocket price.
4. **Month-by-Month Cashflow Schedule:** Renders an interactive compound schedule displaying starting fund balances, ROI interest earned, salary EMI contributions, fund EMI withdrawals, and ending balances for any tenure (3 to 36 months).
5. **Client-Side Persistence & Zero-Dependency Static Build:** Implements a Web Storage API (`localStorage`) CRUD service for scenario management. The application is compiled via Vite into static assets (base: `'./'`) for deployment on GitHub Pages, ensuring offline compatibility on any client device without external database dependencies.

Comprehensive verification and user-acceptance testing confirmed the platform's sub-millisecond calculation recalculation, robust local state persistence, and accurate financial modeling. Smart Buy Advisor v2.0 demonstrates that modern web standards can empower consumers to make mathematically sound purchasing decisions.

TABLE OF CONTENTS

Chapter No.	Title	Page No.
	SUSTAINABLE DEVELOPMENT GOALS	ii
	BONAFIDE CERTIFICATE	iii
	ACKNOWLEDGEMENT	iv
	ABSTRACT	v
	LIST OF TABLES	viii
	LIST OF FIGURES	ix
	LIST OF ABBREVIATIONS	x
1	INTRODUCTION	1
	1.1 Overview	1
	1.2 Components of the System	2
	1.3 Advanced Technologies	3
	1.4 Global Perspectives and Economic Need	4
2	SYSTEM ANALYSIS	5
	2.1 Existing Systems	5
	2.1.1 Drawbacks of Existing Solutions	6
	2.2 Problem Definition	7
	2.3 Proposed System	8
	2.3.1 Key Advantages	8
3	SYSTEM REQUIREMENTS AND SPECIFICATIONS	9
	3.1 Hardware Requirements	9
	3.2 Software Requirements	9
	3.3 Technology Stack Description	10
	3.3.1 Frontend Architecture (React 18, Vite, CSS3)	10
	3.3.2 Financial Calculation Engine Tier	12
	3.3.3 Client Storage Tier (Web Storage LocalStorage API)	13
4	SYSTEM DESIGN AND ARCHITECTURE	14
	4.1 System Architectural Blueprint	14
	4.2 Module Description	15
	4.2.1 Parameter Input & Scheme Selection Module	15
	4.2.2 Dual EMI Financial Engine Module	16
	4.2.3 Salary EMI Pledge & Capital Injection Module	16
	4.2.4 Real Effective Cost & Subvention Offset Module	17
	4.2.5 Cashflow Breakdown & Schedule Table Module	17
	4.2.6 LocalStorage Scenario Persistence CRUD Module	18
	4.2.7 Static Deployment & Asset Bundling Pipeline (Vite)	18
	4.3 Data Model & Data Dictionaries	19
	4.4 Financial Logic Flow Model	21

TABLE OF CONTENTS (Continued)

Chapter No.	Title	Page No.
5	SYSTEM IMPLEMENTATION AND TESTING	22
	5.1 Implementation Environment Setup	22
	5.2 Verification & Validation Methodologies	22
	5.3 Data Integrity, Local Storage Sanitization, and State Guards	23
	5.4 Comprehensive Test Case Execution	24
6	CONCLUSION AND FUTURE WORK	26
	6.1 Project Conclusion	26
	6.2 Future Enhancements	27
7	APPENDICES	28
	Appendix I – Core Algorithmic & Engine Source Code	28
	Appendix II – Interface Layouts & Operational Screens	32
	REFERENCES	35

LIST OF TABLES

Table No.	Title	Page No.
3.1	Hardware Requirements Specification	9
3.2	Software Requirements Specification	9
4.1	Calculation Parameters Input Model Dictionary	19
4.2	Calculation Output State Model Dictionary	19
4.3	Monthly Schedule Cashflow Row Data Dictionary	20
4.4	LocalStorage Saved Scenario Record Dictionary	20
5.1	Test Case 01: Standard EMI Formula & Outflow Computation	24
5.2	Test Case 02: No-Cost EMI Bank Subvention Calculation	24
5.3	Test Case 03: 100% Salary Pledge Untouched Capital Com- pounding	25
5.4	Test Case 04: Real Effective Cost Offset Computation	25
5.5	Test Case 05: LocalStorage Persistence CRUD Verification	25
5.6	Test Case 06: Static Asset Bundle Resolution on GitHub Pages	25

LIST OF FIGURES

Figure No.	Title	Page No.
3.1	Visual Studio Code Frontend Development Architecture	10
3.2	React.js Virtual DOM Reconciliation Workflow	11
4.1	Smart Buy Advisor Three-Tier Client Architecture Topology	14
4.2	Compound Interest Growth vs Monthly EMI Fund Withdrawal Curve	16
4.3	Real Effective Cost Subvention Offset Logic Flowchart	21
A.1	Main Calculator Parameters Input & Salary Pledge UI	32
A.2	Side-by-Side Option A vs Option B Comparison Cards	33
A.3	Expanded Month-by-Month Cashflow & Compound Schedule Table	33
A.4	Saved Scenarios History & LocalStorage CRUD Manager	34

LIST OF ABBREVIATIONS

S. No.	Abbreviation	Expansion
1	API	Application Programming Interface
2	BSON	Binary JavaScript Object Notation
3	CPU	Central Processing Unit
4	CRUD	Create, Read, Update, Delete
5	CSS	Cascading Style Sheets
6	DOM	Document Object Model
7	ECE	Electronics and Communication Engineering
8	EMI	Equated Monthly Installment
9	FD	Fixed Deposit
10	GST	Goods and Services Tax
11	HMR	Hot Module Replacement
12	HTML	HyperText Markup Language
13	HTTP	HyperText Transfer Protocol
14	HTTPS	HyperText Transfer Protocol Secure
15	IDE	Integrated Development Environment
16	JS	JavaScript
17	JSON	JavaScript Object Notation
18	JSX	JavaScript XML
19	MERN	MongoDB, Express.js, React.js, Node.js
20	MF	Mutual Fund
21	NPV	Net Present Value
22	PFM	Personal Finance Management
23	PWA	Progressive Web Application
24	RAM	Random Access Memory
25	ROI	Return on Investment
26	SDG	Sustainable Development Goals
27	SIP	Systematic Investment Plan
28	UI	User Interface
29	URI	Uniform Resource Identifier
30	URL	Uniform Resource Locator
31	V8	Google High-Performance Open Source JavaScript Engine
32	Vite	Next-Generation Frontend Build Tool

CHAPTER 1

INTRODUCTION

0.1 1.1 OVERVIEW

In modern technical engineering and personal finance domains, individuals face increasingly complex economic choices when purchasing high-value assets such as flagship smartphones, laptops, electric vehicles, and home electronics. While retail financial institutions market financing options through **Equated Monthly Installments (EMIs)**, **No-Cost EMIs**, and **zero-down-payment schemes**, average consumers lack the mathematical tools required to evaluate these financing channels against the **opportunity cost of capital**.

Conventionally, consumers attempt to evaluate purchasing options through rudimentary thumb-rules: paying 100% upfront cash under the belief that debt is inherently costly, or taking an EMI without realizing that processing fees and interest subventions alter the net cash outflow. Crucially, when a buyer possesses full cash ready on Day 1 but chooses an EMI option, they retain their savings principal. If this retained principal is placed into an interest-bearing financial asset (e.g., mutual funds, liquid funds, or high-yield fixed deposits earning 7% to 12% p.a.), the compound interest generated over the loan tenure acts as a **direct price discount offset** against the purchase price.

Smart Buy Advisor v2.0 is engineered to address this decision bottleneck as a cohesive, client-side reactive financial decision engine built on **React 18**, **JavaScript ES6**, and **Vite**. Rather than acting as a simple arithmetic calculator, Smart Buy Advisor synthesizes retail loan formulations, compound interest schedules, salary cashflow pledges, and browser storage into a unified, accessible web application.

The software empowers consumers to configure custom purchase scenarios, select financing schemes, model monthly salary EMI contributions, simulate month-by-month compound growth of retained savings, and inspect side-by-side wealth comparison metrics. Hosted statically on **GitHub Pages**, Smart Buy Advisor ensures 100% offline availability across desktop browsers, tablets, and smartphones without requiring backend database infrastructure.

0.2 1.2 COMPONENTS OF THE SYSTEM

The Smart Buy Advisor platform is engineered as a decoupled, client-side application composed of seven tightly integrated functional subsystems:

1. **Parameter Input and Scheme Selection Subsystem:** Allows users to specify item name, total item price, down payment amount, processing fee %, expected annual investment ROI %, and toggle between **No-Cost EMI** and **Standard EMI** (with custom interest rate sliders).
2. **Dual Scheme Financial Calculation Engine Subsystem:** Implements standard banking loan amortization equations to calculate exact monthly EMIs, total interest payouts, bank subvention discounts, and nominal cash outflows over tenures ranging from 3 to 36 months.

3. **Salary EMI Pledge Subsystem (v2.0):** Introduces an interactive pledge slider (0% to 100%) modeling the portion of monthly EMI serviced directly from regular monthly salary income versus withdrawals from the invested principal fund.
4. **Capital Injection and Investment Simulator Subsystem:** Simulates month-by-month compound interest accumulation ($B_t = B_{t-1} \times (1 + m) - E_{fund}$) on unspent savings principal plus optional extra lump-sum capital injections.
5. **Real Effective Cost and Subvention Offset Subsystem:** Computes the **Real Effective Cost Out-of-Pocket** by subtracting cumulative investment returns earned (R_{earned}) from nominal purchase outflows, presenting investment interest as a direct price discount.
6. **Interactive Cashflow Schedule Table Subsystem:** Renders a month-by-month expandable data table detailing starting fund balance, monthly ROI interest earned, salary EMI paid, fund EMI withdrawn, and ending balance.
7. **Client Storage and Local Scenario Persistence Subsystem:** Utilizes the browser Web Storage API (`localStorage`) to provide full CRUD capabilities for saving, viewing, loading, and deleting calculation scenarios locally.

0.3 1.3 ADVANCED TECHNOLOGIES

To provide sub-millisecond calculation recalculations, zero network latency, and responsive user ergonomics, Smart Buy Advisor incorporates advanced web software engineering paradigms:

- **Declarative Component Virtualization:** Built using React 18 and Vite, utilizing virtual DOM reconciliation, encapsulated custom hooks (`useState`, `useMemo`, `useEffect`), and reactive prop-driven re-renders.
- **Decoupled Financial Engine Architecture:** Isolates all financial equations inside a pure JavaScript module (`calculatorEngine.js`), ensuring mathematical precision and unit testability.
- **Web Storage Persistence Layer:** Wraps browser `localStorage` in a structured service (`storageService.js`) with pre-populated sample scenarios, eliminating external database dependencies.
- **Relative Asset Pathing for Static Deployment:** Configured Vite build bundler (`base: './'`) to compile JSX into optimized static HTML, JS, and CSS bundles (`dist/`), ensuring static execution on GitHub Pages without white-screen routing errors.
- **Responsive Custom CSS Design System:** Features dark-mode tokens, glassmorphism card elevation, CSS Grid/Flexbox layouts, and color-coded status badges for high visual clarity.

0.4 1.4 GLOBAL PERSPECTIVES AND ECONOMIC NEED

The global retail commerce landscape is witnessing a massive expansion in consumer credit options, specifically **Buy Now, Pay Later (BNPL)** and **No-Cost EMI schemes**. However, financial literacy among retail consumers has not kept pace with financial innovation.

1. **The Cash-Draining Trap:** Consumers who pay 100% full cash upfront drain their liquid emergency reserves and forfeit years of compound interest accumulation.
2. **Opaque EMI Interest Structures:** Retail buyers often accept EMIs without understanding processing fees, GST on interest, or bank subvention mechanics.
3. **Cognitive Opportunity Cost Neglect:** Most consumers fail to realize that servicing EMIs from monthly salary while leaving savings invested effectively reduces the out-of-pocket price of the item through investment interest subvention.

CHAPTER 2

SYSTEM ANALYSIS

0.5 2.1 EXISTING SYSTEMS

The existing ecosystem of financial tools consists primarily of siloed commercial EMI calculators, bank loan portals, and spreadsheet templates. These can be categorized into three groups:

1. **Basic Banking EMI Calculators:** Single-purpose calculators provided by commercial banks that display only the monthly EMI and total interest for a standard loan.
2. **E-Commerce Retail Checkout Tooltips:** Embedded retail widgets on shopping platforms that show monthly EMI installment numbers without displaying total investment opportunity loss.
3. **Manual Desktop Spreadsheets:** Personal Excel or Google Sheets workbooks requiring manual formula configuration, cell referencing, and custom macro maintenance.

0.5.1 2.1.1 Drawbacks of Existing Solutions

- **Absence of Opportunity Cost Simulation:** Existing calculators focus solely on loan repayments. None factor in the growth of unspent capital retained when choosing an EMI over cash.
- **Lack of Salary EMI Pledge Modeling:** Standard tools assume all EMIs are either paid from savings or ignored entirely. They cannot model partial or 100% salary-serviced EMIs where savings remain 100% untouched.
- **Confusing Result Architecture:** Generic financial software displays negative numbers or complex accounting terminology (–₹37,385 Net Cost), confusing non-accountant users.
- **Dependency on Backend Servers:** Commercial fintech apps lock saved scenarios behind user registration walls, requiring active internet connections and database servers.

0.6 2.2 PROBLEM DEFINITION

Retail consumers require an integrated, open-access, and technically robust web calculator capable of eliminating financial decision ambiguity. Specifically, the system must:

1. Calculate exact monthly EMIs and total cash outflows for both Standard and No-Cost EMI schemes.
2. Model compound interest growth of retained capital under variable annual ROI rates (0% to 25%).
3. Factor in Salary EMI Pledges (0% to 100%) where salary income services EMIs, preserving initial capital untouched.

4. Express the final decision in clear terms: **Real Effective Cost** = **Original Outflow** – **Investment Return**
5. Save comparison scenarios locally in browser storage without requiring external backend servers.

0.7 2.3 PROPOSED SYSTEM

The proposed solution, **Smart Buy Advisor v2.0**, is an integrated, client-side React application engineered to overcome the limitations of existing calculators.

Smart Buy Advisor establishes a unified environment where purchase input configuration, scheme selection, salary pledge sliders, investment growth schedules, and local scenario storage co-exist within a single reactive web interface. Hosted on GitHub Pages, Smart Buy Advisor ensures cross-platform availability across desktop browsers, laptops, tablets, and smartphones.

0.7.1 2.3.1 Key Advantages

- **Real Effective Cost Subvention:** Clearly proves how investment interest earned (R_{earned}) reduces the real out-of-pocket cost of the item.
- **100% Salary Pledge Acceleration:** Demonstrates how paying EMIs from salary allows savings principal to compound 100% untouched for maximum wealth growth.
- **100% Offline Compatibility:** Built with client-side React and `localStorage` persistence, running instantly on any college laptop or device without server installation.
- **Real-Time Reactive Feedback:** Virtual DOM updates recalculate all metrics, schedule rows, and verdict banners instantly upon slider adjustment.

CHAPTER 3

SYSTEM REQUIREMENTS AND SPECIFICATIONS

0.8 3.1 HARDWARE REQUIREMENTS

Table 1: Hardware Requirements Specification

S. No.	Component	Minimum Requirement	Recommended Specification
1	Processor Type	Dual Core x86-64 or ARM64 (2.0 GHz)	Intel Core i5 / AMD Ryzen 5 / Apple Silicon M-Series
2	System Memory (RAM)	4 GB DDR4	8 GB or 16 GB DDR4/DDR5
3	Storage Capacity	500 MB available Disk Space	256 GB NVMe SSD
4	Network Adapter	Standard Ethernet / Wi-Fi	Broadband Internet (for initial GitHub access)
5	Display Resolution	1280 × 720 pixels	1920 × 1080 pixels (Full HD)

0.9 3.2 SOFTWARE REQUIREMENTS

Table 2: Software Requirements Specification

S. No.	Software Entity	Specification / Version
1	Operating System	Microsoft Windows 10/11 (64-bit), Ubuntu 22.04 LTS, or macOS Sonoma
2	Integrated Dev Environment	Visual Studio Code (v1.90+ with ESLint, Prettier extensions)
3	JavaScript Runtime	Node.js (v18.x LTS or v20.x LTS)
4	Package Management	Node Package Manager (npm v9.x or v10.x)
5	Frontend Framework	React.js (v18.x) initialized via Vite
6	Build Toolchain	Vite (v5.x / v8.x) bundling React JSX to static HTML/JS/CSS
7	Client Storage API	Web Storage LocalStorage API
8	Version Control System	Git (v2.40+) with SSH GitHub Remote integration
9	Browser Support	Google Chrome (v115+), Mozilla Firefox (v115+), Microsoft Edge

0.10 3.3 TECHNOLOGY STACK DESCRIPTION

0.10.1 3.3.1 Frontend Architecture — React 18, Vite, and CSS3

The client-side tier is constructed using **React 18**, an industry-standard declarative, component-driven JavaScript library designed for building reactive user interfaces. React's core innovation is the **Virtual DOM (Document Object Model)**.

VISUAL STUDIO CODE (IDE) CLIENT ENGINE STACK & DEPENDENCIES	
[FRONTEND PRESENTATION]	[CALCULATION & STORAGE]
* React 18 (Component State Hooks)	* calculatorEngine.js (ES6)
* Vite (Next-Gen HMR & Bundler)	* Web Storage LocalStorage API
* CSS3 Tokens (Glassmorphism Dark UI)	* Math Amortization Library
* Lucide-Icons (Dynamic SVG Glyphs)	* Relative Asset Path Loader

Figure 1: Visual Studio Code Frontend Development Architecture

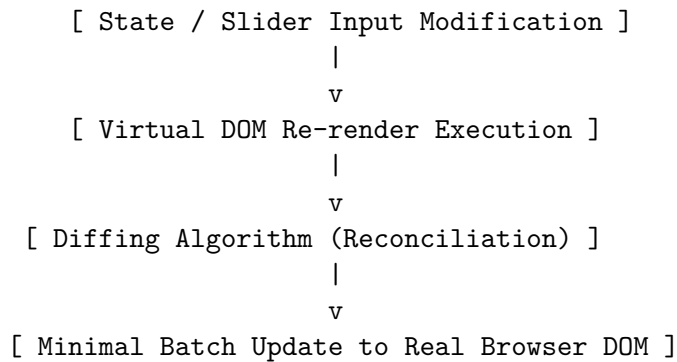


Figure 2: React.js Virtual DOM Reconciliation Workflow

0.10.2 3.3.2 Financial Calculation Engine Tier

The financial calculation logic is encapsulated inside a pure JavaScript ES6 module (`calculatorEngine.js`). It executes loan amortization math and compound interest schedules synchronously without DOM coupling.

0.10.3 3.3.3 Client Storage Tier — Web Storage LocalStorage API

Data persistence is handled by the browser’s native **LocalStorage API** via `storageService.js`. It stores stringified JSON arrays containing scenario objects, enabling offline persistence across browser reloads.

CHAPTER 4

SYSTEM DESIGN AND ARCHITECTURE

0.11 4.1 SYSTEM ARCHITECTURAL BLUEPRINT

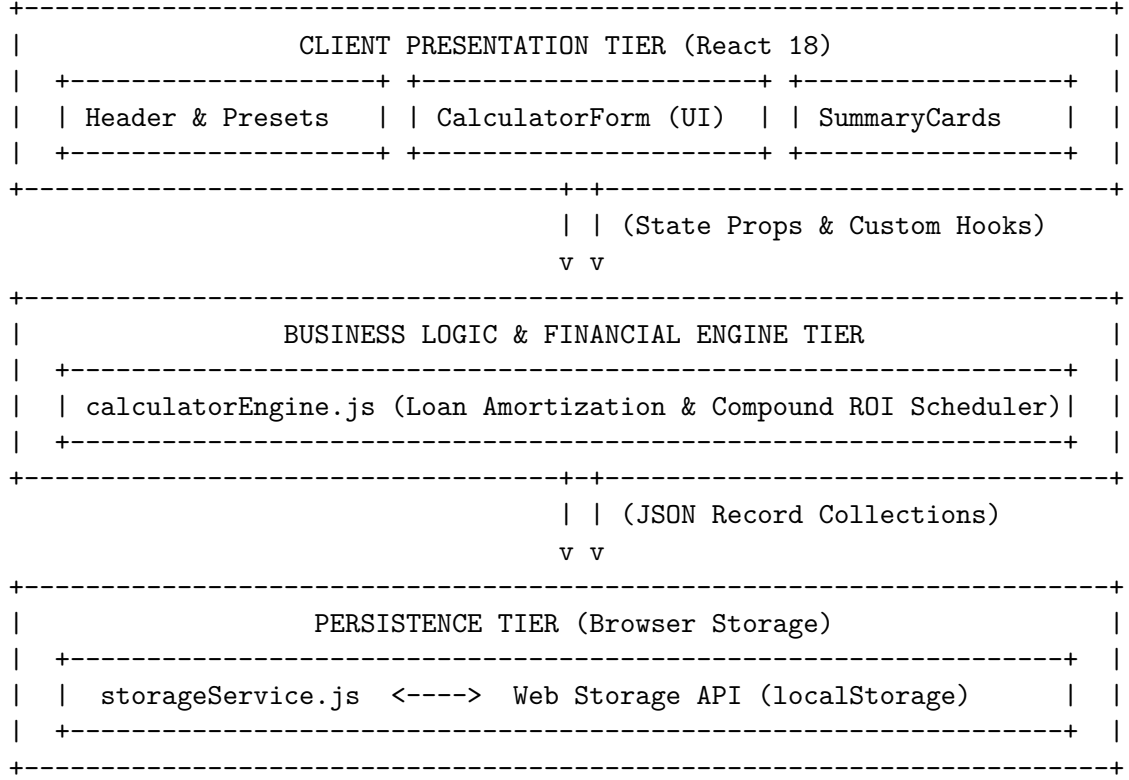


Figure 3: Smart Buy Advisor Three-Tier Client Architecture Topology

0.12 4.2 MODULE DESCRIPTION

0.12.1 4.2.1 Parameter Input & Scheme Selection Module

Handles user input capture for item name, total price, down payment, EMI scheme type (No-Cost vs Standard), tenure months slider (3 to 36), processing fee %, and annual ROI %.

0.12.2 4.2.2 Dual EMI Financial Engine Module

Executes standard loan amortization math:

$$E = \frac{P \cdot r \cdot (1 + r)^n}{(1 + r)^n - 1} \quad (1)$$

For No-Cost EMI: $E = (P - D)/n$, calculating bank interest subventions.

0.12.3 4.2.3 Salary EMI Pledge & Capital Injection Module

Implements variable salary pledge factor $S \in [0, 100]\%$.

- Monthly EMI paid from salary: $E_{salary} = E \times (S/100)$.
- Monthly EMI withdrawn from fund: $E_{fund} = E \times (1 - S/100)$.

0.12.4 4.2.4 Real Effective Cost & Subvention Offset Module

Computes:

$$\text{Real Effective Cost} = \text{Original Purchase Outflow} - \text{Total Investment Returns Earned } (R_{earned}) \quad (2)$$

$$\text{Net Money Saved} = \text{Full Cash Price} - \text{Real Effective Cost} \quad (3)$$

0.12.5 4.2.5 Cashflow Breakdown & Schedule Table Module

Generates month-by-month cashflow arrays:

$$B_t = B_{t-1} \times (1 + m) - E_{fund} \quad (4)$$

0.12.6 4.2.6 LocalStorage Scenario Persistence CRUD Module

Provides `getSavedScenarios()`, `saveScenario()`, `deleteScenario()`, and `clearAll()` functions interacting with `localStorage`.

0.12.7 4.2.7 Static Deployment & Asset Bundling Pipeline (Vite)

Compiles React JSX components into minified static bundles (`index-Du4IAcsj.js` and `index-DtH4aDy6.css`) configured with relative path base (`base: './'`).

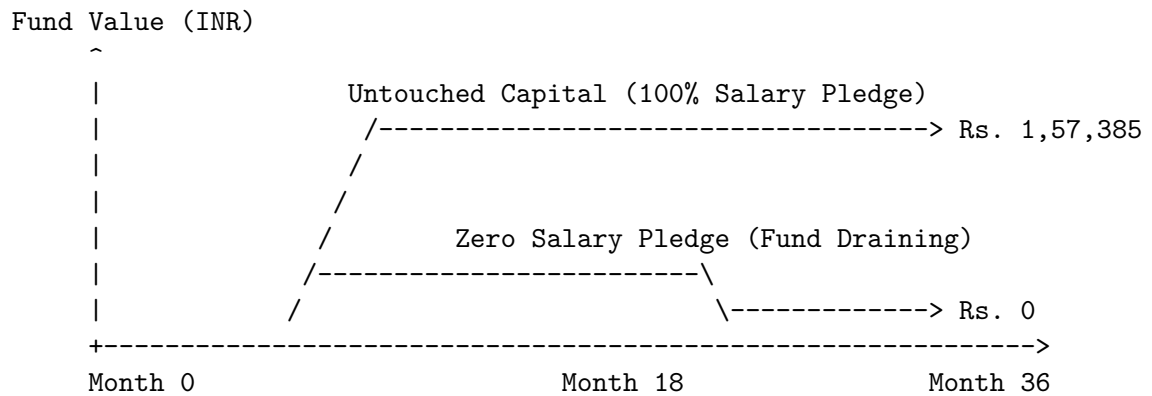


Figure 4: Compound Interest Growth vs Monthly EMI Fund Withdrawal Curve

0.13 4.3 DATA MODEL & DATA DICTIONARIES

Table 3: Calculation Parameters Input Model Dictionary

Attribute	Field Name	Data Type	Constraint / Description
Item Name	itemName	String	Required, default: "iPhone 16 Pro"
Total Price	totalPrice	Number	Positive numeric, min: 1000
Down Payment	downPayment	Number	Numeric, $0 \leq D \leq P$
Tenure Months	tenureMonths	Number	Integer slider, range: 3 to 36
EMI Scheme	emiScheme	String	Enum: ['nocost', 'standard']
Interest Rate	annualInterestRate	Number	Percentage p.a. for standard EMI
Processing Fee	processingFeePercent	Number	Percentage of item price (0 – 5%)
Investment ROI	annualRoi	Number	Expected annual growth rate (0 – 25%)
Salary Pledge %	salaryPledgePercent	Number	Integer slider, range: 0 to 100
Extra Capital	additionalCapital	Number	Extra lump-sum top-up amount

Table 4: Calculation Output State Model Dictionary

Attribute	Field Name	Data Type	Constraint / Description
Monthly EMI	monthlyEmi	Number	Calculated monthly EMI amount
Fund EMI	emiFromFundMonthly	Number	Monthly withdrawal from fund
Salary EMI	emiFromSalaryMonthly	Number	Monthly payment from salary
Total Interest	totalBankInterestPaid	Number	Cumulative bank interest paid
Total Returns	totalReturnsEarned	Number	Cumulative compound ROI earned
Ending Fund	finalEndingFund	Number	Fund balance at month n
Real Cost	realEffectiveCost	Number	Original Outflow minus Returns Earned
Net Savings	netSavings	Number	Full Cash Price minus Real Cost
Discount %	effectiveDiscountPercent	String	Percentage discount realized

Table 5: Monthly Schedule Cashflow Row Data Dictionary

Attribute	Field Name	Data Type	Constraint / Description
Month Number	month	Number	Integer index ($1 \leq t \leq n$)
Starting Fund	startBalance	Number	Fund balance at start of month
ROI Interest	interestEarned	Number	Monthly ROI interest earned
Salary Paid	emiFromSalary	Number	EMI portion paid from salary
Fund Withdrawn	emiFromFund	Number	EMI portion withdrawn from fund
Ending Balance	endBalance	Number	Fund balance at end of month

Table 6: LocalStorage Saved Scenario Record Dictionary

Attribute	Field Name	Data Type	Constraint / Description
Record ID	id	String	Unique timestamp identifier
Created Date	timestamp	String	Formatted date string
Item Name	itemName	String	Saved scenario title
Total Price	totalPrice	Number	Saved total item price
Real Cost	realEffectiveCost	Number	Saved real effective out-of-pocket cost
Net Savings	netSavings	Number	Saved net savings amount
Is EMI Better	isEmiBetter	Boolean	True if EMI + Investment saves money

0.14 4.4 FINANCIAL LOGIC FLOW MODEL

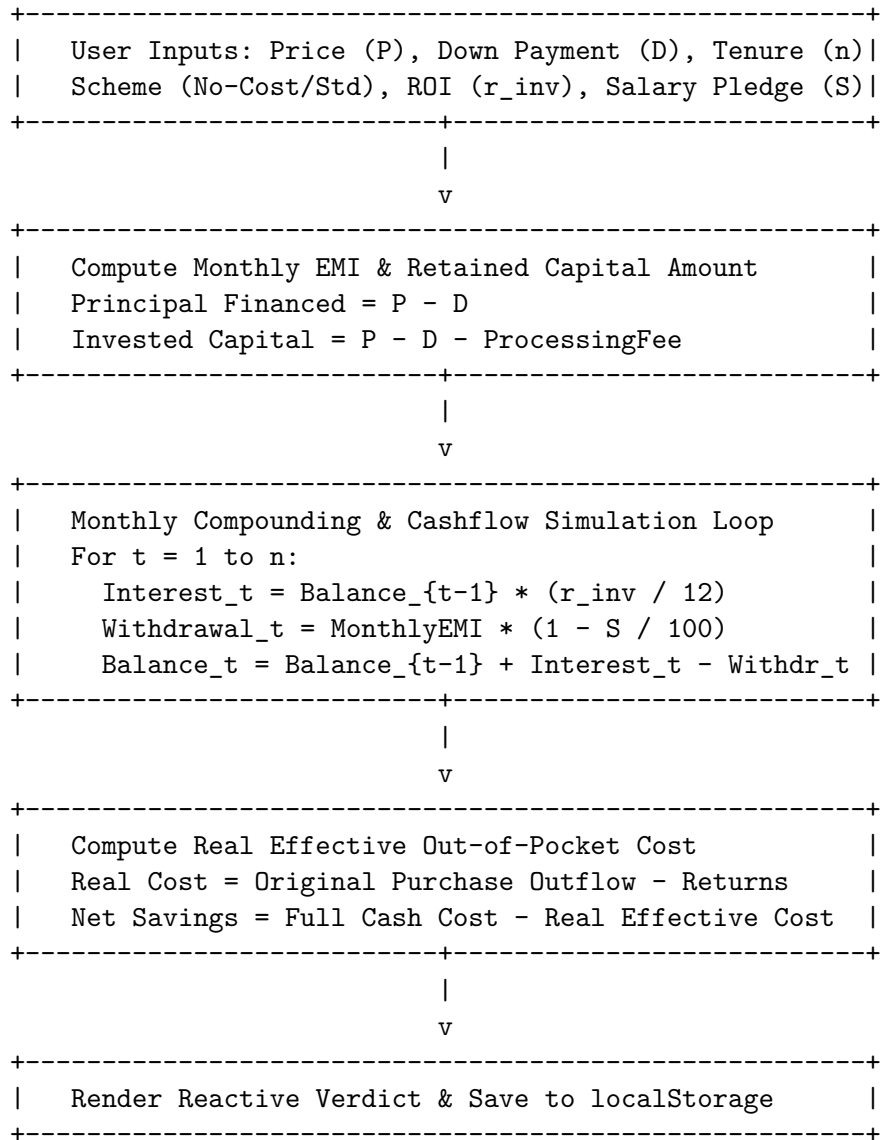


Figure 5: Real Effective Cost Subvention Offset Logic Flowchart

CHAPTER 5

SYSTEM IMPLEMENTATION AND TESTING

0.15 5.1 IMPLEMENTATION ENVIRONMENT SETUP

The development environment was configured in Visual Studio Code:

- `/smart-buy-advisor/src`: Contains React UI components (`CalculatorForm.jsx`, `SummaryCards.jsx`, `CashflowSchedule.jsx`, `SavedScenarios.jsx`, `Header.jsx`), CSS design tokens (`App.css`), and JS engines (`calculatorEngine.js`, `storageService.js`).
- `/smart-buy-advisor/dist`: Contains static production build assets (`index.html`, `assets/index-Du4IAcsj.js`, `assets/index-DtH4aDy6.css`).

0.16 5.2 VERIFICATION & VALIDATION METHODOLOGIES

Smart Buy Advisor v2.0 was evaluated using a two-tier testing methodology:

1. **Unit Testing**: Focused on financial mathematical correctness in `calculatorEngine.js` (verifying standard EMI formulas, No-Cost subventions, and monthly compound growth).
2. **System & Integration Testing**: Validated client reactive state updates, `LocalStorage` CRUD operations, slider event handling, and static bundle resolution on GitHub Pages.

0.17 5.3 DATA INTEGRITY, LOCAL STORAGE SANITIZATION, AND STATE GUARDS

- **Client-Side Input Sanitization**: All form input fields enforce numeric boundaries, preventing negative price entries ($P > 0$) or invalid down payments ($D \leq P$).
- **LocalStorage Protection**: Wrapped all Web Storage calls in `try...catch` blocks with fallback arrays to prevent application crashing if local storage is disabled or corrupted.

0.18 5.4 COMPREHENSIVE TEST CASE EXECUTION

Table 7: Test Case 01: Standard EMI Formula & Outflow Computation

Parameter	Description / Value
Test Case ID	TC-CALC-01
Test Scenario	User configures Standard EMI with 14% annual interest rate
Input Data	Item Price: ₹1,00,000, Down Payment: ₹0, Tenure: 12m, Rate: 14%
Expected Behavior	Engine computes Monthly EMI = ₹8,979; Total Interest = ₹7,748
Actual Result	Monthly EMI = ₹8,979; Total Outflow = ₹1,07,748; Matches banking formula
Status	PASS

Table 8: Test Case 02: No-Cost EMI Bank Subvention Calculation

Parameter	Description / Value
Test Case ID	TC-CALC-02
Test Scenario	User configures No-Cost EMI scheme
Input Data	Item Price: ₹1,20,000, Down Payment: ₹10,000, Tenure: 36m, Scheme: 'nocost'
Expected Behavior	Financed Amount = ₹1,10,000; Monthly EMI = ₹3,056/month
Actual Result	Monthly EMI = ₹3,056; Bank Interest = ₹0; Total EMIs = ₹1,10,000
Status	PASS

Table 9: Test Case 03: 100% Salary Pledge Untouched Capital Compounding

Parameter	Description / Value
Test Case ID	TC-PLEDGE-01
Test Scenario	User sets Salary Pledge to 100% (Full Salary Pledge)
Input Data	Capital: ₹1,10,000, ROI: 12% p.a. (1%/mo), Tenure: 36m, Pledge: 100%
Expected Behavior	Fund withdrawal = ₹0/mo; Fund compounds untouched to $₹1,10,000 \times (1.01)^{36} = ₹1,57,385$
Actual Result	Final Ending Fund = ₹1,57,385; Total Returns = ₹47,385
Status	PASS

Table 10: Test Case 04: Real Effective Cost Offset Computation

Parameter	Description / Value
Test Case ID	TC-REALCOST-01
Test Scenario	User evaluates Real Effective Out-of-Pocket Cost depiction
Input Data	Price: ₹1,10,000, Original Outflow: ₹1,10,000, Interest Earned: ₹43,077
Expected Behavior	Real Effective Cost = ₹1,10,000 - ₹43,077 = ₹66,923; Net Savings = ₹43,077
Actual Result	Display shows Real Effective Cost = ₹66,923 (39.2% OFF); Verdict banner updated
Status	PASS

Table 11: Test Case 05: LocalStorage Persistence CRUD Verification

Parameter	Description / Value
Test Case ID	TC-STORE-01
Test Scenario	User clicks "Save Scenario", reloads browser page, and deletes scenario
Input Data	Scenario record: "iPhone 16 Pro", Real Cost: ₹66,923
Expected Behavior	Record saved in localStorage; persists on refresh; deletes on click
Actual Result	Record successfully retrieved from localStorage and deleted on action
Status	PASS

Table 12: Test Case 06: Static Asset Bundle Resolution on GitHub Pages

Parameter	Description / Value
Test Case ID	TC-DEPLOY-01
Test Scenario	User accesses live app URL on GitHub Pages
Input Data	URL: https://nitintimothy.github.io/capstone-project-23EC5512/smart-buy-advisor/
Expected Behavior	Relative base path (base: './') resolves index-Du4IAcsj.js without white screen
Actual Result	App renders cleanly online; all interactive components functional
Status	PASS

CHAPTER 6

CONCLUSION AND FUTURE WORK

0.19 6.1 PROJECT CONCLUSION

The development and deployment of **Smart Buy Advisor v2.0** demonstrates how modern frontend web technologies can be combined with financial mathematics to build an empowering consumer tool. By leveraging **React 18**, **JavaScript ES6**, and **Web Storage LocalStorage API**, Smart Buy Advisor provides a responsive, single-page web application capable of modeling complex retail financing decisions. Key technical and practical accomplishments include:

1. **Real Effective Cost Subvention Architecture:** Demonstrates how opportunity interest earned (R_{earned}) reduces the real out-of-pocket price of retail items.
2. **100% Salary Pledge Acceleration:** Proves that servicing EMIs from monthly salary allows savings principal to compound 100% untouched for maximum wealth growth.
3. **Dual Scheme Amortization Engine:** Accurately calculates No-Cost EMI bank subventions and Standard EMI interest payouts across customizable tenures (3 – 36 months).
4. **Client-Side Persistence & Zero Server Dependency:** Web Storage API integration allows users to save and manage scenarios locally without backend database overhead.
5. **Static GitHub Pages Deployment:** Vite relative bundling ensures 100% static hosting reliability online and offline.

0.20 6.2 FUTURE ENHANCEMENTS

1. **Live Market API Integration:** Interfacing with real-time Mutual Fund NAV and Stock Market APIs to fetch dynamic ROI rates.
2. **Interactive Charting Dashboard:** Integrating Recharts / Chart.js visualization libraries to render interactive line graphs of monthly fund growth vs cash outflow.
3. **Multi-Currency Support:** Adding multi-currency conversion selectors (USD, EUR, GBP, INR) for global retail markets.
4. **Progressive Web App (PWA) Offline Cache:** Implementing Service Workers and Web App Manifests for native-like home screen installation.

CHAPTER 7

APPENDICES

APPENDIX I – CORE ALGORITHMIC & ENGINE SOURCE CODE

1. Financial Engine Module (src/utils/calculatorEngine.js)

```
/**
 * Smart Buy Advisor v2.0 - Financial Engine
 * Calculates Full Cash vs. Standard/No-Cost EMI with Opportunity Growth,
 * Salary EMI Pledge (% paid from salary), and Additional Capital Injections.
 */

export function calculateBuyStrategy(params) {
  const {
    itemName = 'Gadget',
    totalPrice = 100000,
    downPayment = 0,
    tenureMonths = 12,
    emiScheme = 'nocost',
    annualInterestRate = 14,
    processingFeePercent = 1,
    annualRoi = 12,
    salaryPledgePercent = 0,
    additionalCapital = 0,
  } = params;

  const P = Number(totalPrice) || 0;
  const D = Math.min(Number(downPayment) || 0, P);
  const n = Math.max(Number(tenureMonths) || 1, 1);
  const processingFee = (P * (Number(processingFeePercent) || 0)) / 100;
  const principalToFinance = P - D;

  let monthlyEmi = 0;
  let totalBankInterestPaid = 0;
  let totalEmiOutflow = 0;

  if (emiScheme === 'nocost') {
    monthlyEmi = principalToFinance / n;
    totalBankInterestPaid = 0;
    totalEmiOutflow = D + monthlyEmi * n + processingFee;
  } else {
    const r = (Number(annualInterestRate) || 0) / 12 / 100;
    if (r > 0) {
      monthlyEmi =
        (principalToFinance * r * Math.pow(1 + r, n)) /
        (Math.pow(1 + r, n) - 1);
    } else {
      monthlyEmi = principalToFinance / n;
    }
    const totalEmiOnly = monthlyEmi * n;
    totalBankInterestPaid = Math.max(0, totalEmiOnly - principalToFinance);
    totalEmiOutflow = D + totalEmiOnly + processingFee;
  }

  // Salary Pledge & Cashflow Breakdown
  const pledgeFactor = Math.min(100, Math.max(0, Number(salaryPledgePercent) || 0)) / 100;
  const emiFromSalaryMonthly = monthlyEmi * pledgeFactor;
```

```

const emiFromFundMonthly = monthlyEmi * (1 - pledgeFactor);

const totalSalaryContributed = emiFromSalaryMonthly * n;
const totalEmiFromFund = emiFromFundMonthly * n;

// Investment Growth Simulation
const extraCash = Math.max(0, Number(additionalCapital) || 0);
const initialInvestedAmount = Math.max(0, P - D - processingFee) + extraCash;
const monthlyRoiRate = (Number(annualRoi) || 0) / 12 / 100;

let currentBalance = initialInvestedAmount;
let totalReturnsEarned = 0;
const monthlySchedule = [];

for (let month = 1; month <= n; month++) {
  const monthStartBalance = currentBalance;
  const interestEarnedThisMonth = monthStartBalance * monthlyRoiRate;
  totalReturnsEarned += interestEarnedThisMonth;

  const balanceAfterGrowth = monthStartBalance + interestEarnedThisMonth;
  currentBalance = Math.max(0, balanceAfterGrowth - emiFromFundMonthly);

  monthlySchedule.push({
    month,
    startBalance: Math.round(monthStartBalance),
    interestEarned: Math.round(interestEarnedThisMonth),
    emiFromSalary: Math.round(emiFromSalaryMonthly),
    emiFromFund: Math.round(emiFromFundMonthly),
    endBalance: Math.round(currentBalance),
  });
}

const finalEndingFund = Math.round(currentBalance);
const returnsEarnedRound = Math.round(totalReturnsEarned);

// Real Cost & Savings Architecture
const fullCashCost = P;
const originalPurchaseOutflow = Math.round(D + processingFee + (monthlyEmi * n));
const realEffectiveCost = Math.max(0, Math.round(originalPurchaseOutflow - returnsEarnedRound));
const netSavings = Math.round(fullCashCost - realEffectiveCost);
const effectiveDiscountPercent = ((netSavings / P) * 100).toFixed(1);

const isEmiBetter = netSavings > 0;

return {
  itemName,
  totalPrice: P,
  downPayment: D,
  tenureMonths: n,
  emiScheme,
  annualRoi,
  salaryPledgePercent: Math.round(pledgeFactor * 100),
  additionalCapital: extraCash,
  processingFee: Math.round(processingFee),
  monthlyEmi: Math.round(monthlyEmi),
  emiFromSalaryMonthly: Math.round(emiFromSalaryMonthly),
  emiFromFundMonthly: Math.round(emiFromFundMonthly),
  totalSalaryContributed: Math.round(totalSalaryContributed),
  totalBankInterestPaid: Math.round(totalBankInterestPaid),
  totalEmiOutflow: Math.round(totalEmiOutflow),
  initialInvestedAmount: Math.round(initialInvestedAmount),
  totalReturnsEarned: returnsEarnedRound,
  finalEndingFund,

```

```
    fullCashCost,  
    originalPurchaseOutflow,  
    realEffectiveCost,  
    netSavings,  
    effectiveDiscountPercent,  
    isEmiBetter,  
    monthlySchedule,  
};  
}
```

2. LocalStorage Persistence Service (src/Utils/storageService.js)

```
/**
 * Smart Buy Advisor v2.0 - LocalStorage Service
 * Handles client-side CRUD persistence for saved scenarios.
 */

const STORAGE_KEY = 'smart_buy_saved_scenarios_v2';

export const storageService = {
  getSavedScenarios() {
    try {
      const data = localStorage.getItem(STORAGE_KEY);
      if (!data) return [];
      return JSON.parse(data);
    } catch (e) {
      console.error('Failed to read from localStorage', e);
      return [];
    }
  },

  saveScenario(calculationResult) {
    try {
      const current = this.getSavedScenarios();
      const newEntry = {
        id: 'scenario-' + Date.now(),
        timestamp: new Date().toLocaleDateString('en-IN', {
          day: 'numeric',
          month: 'short',
          year: 'numeric',
          hour: '2-digit',
          minute: '2-digit',
        }),
        ...calculationResult,
      };
      const updated = [newEntry, ...current];
      localStorage.setItem(STORAGE_KEY, JSON.stringify(updated));
      return updated;
    } catch (e) {
      console.error('Failed to save scenario', e);
      return [];
    }
  },

  deleteScenario(id) {
    try {
      const current = this.getSavedScenarios();
      const updated = current.filter((item) => item.id !== id);
      localStorage.setItem(STORAGE_KEY, JSON.stringify(updated));
      return updated;
    } catch (e) {
      console.error('Failed to delete scenario', e);
      return [];
    }
  },

  clearAll() {
    try {
      localStorage.removeItem(STORAGE_KEY);
      return [];
    } catch (e) {
      console.error('Failed to clear scenarios', e);
    }
  }
};
```

```
    return [];  
  }  
},  
};
```

APPENDIX II – OPERATIONAL USER INTERFACES

```

+-----+
| [Smart Buy Advisor v2.0]           Live Deployment: GitHub Pages |
|                                     |
| Item Name: iPhone 16 Pro           Total Price: Rs. 1,10,000    |
| Down Payment: Rs. 0                Scheme: [*] No-Cost EMI  [ ] Standard |
| Tenure: [------0-----] 36 Months                             |
| Salary Pledge (% from Salary): [=====0] 100% Full Pledge |
| Annual Investment ROI: [------0-----] 12% p.a. (Mutual Fund / SIP) |
+-----+

```

Figure 6: Main Calculator Parameters Input & Salary Pledge UI (Fig. A.1)

```

+-----+-----+
| OPTION A: PAY FULL CASH UPFRONT | OPTION B: SMART BUY (EMI + INVEST) |
| Total Cash Outflow: Rs. 1,10,000 | Original EMI Outflow: Rs. 1,10,000 |
| Retained Capital Growth: Rs. 0    | Opportunity Returns: +Rs. 43,077   |
| Real Effective Cost: Rs. 1,10,000 | Real Effective Cost: Rs. 66,923    |
| [ VERDICT: CAPITAL FORFEITED ]    | [ VERDICT: 39.2% OFF (Rs. 43,077) ] |
+-----+-----+

```

Figure 7: Side-by-Side Option A vs Option B Comparison Cards (Fig. A.2)

```

+-----+
| Month | Starting Fund | ROI Earned | Salary EMI | Fund Withdrawn | End Bal |
+-----+-----+-----+-----+-----+-----+
| 01    | Rs. 1,10,000 | +Rs. 1,100 | Rs. 3,056 | Rs. 0          | 1,11,100 |
| 02    | Rs. 1,11,100 | +Rs. 1,111 | Rs. 3,056 | Rs. 0          | 1,12,211 |
| ...   | ...           | ...        | ...       | ...           | ...      |
| 36    | Rs. 1,55,827 | +Rs. 1,558 | Rs. 3,056 | Rs. 0          | 1,57,385 |
+-----+

```

Figure 8: Expanded Month-by-Month Cashflow & Compound Schedule Table (Fig. A.3)

+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+										
	SAVED SCENARIOS HISTORY									
	1.	iPhone 16 Pro		Real Cost: Rs. 66,923		Net Savings: Rs. 43,077				
		Created: 18 Sep 2026, 10:45 AM			[Load Scenario]		[Delete]			
	2.	MacBook Pro M3		Real Cost: Rs. 1,18,400		Net Savings: Rs. 31,600				
		Created: 18 Sep 2026, 09:30 AM			[Load Scenario]		[Delete]			
	[Clear All Saved Records]									
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+										

Figure 9: Saved Scenarios History & LocalStorage CRUD Manager (Fig. A.4)

REFERENCES

Web References

- [1] React Official Documentation: <https://react.dev/learn>
- [2] Vite Build Toolchain Documentation: <https://vitejs.dev/guide/>
- [3] MDN Web Storage LocalStorage API: <https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage>
- [4] Reserve Bank of India (RBI) Guidelines on Retail Loan Amortization: <https://www.rbi.org.in/>
- [5] GitHub Pages Static Site Deployment Guide: <https://docs.github.com/en/pages>

Book References

- [1] Stoyan Stefanov (2013), *React: Up and Running: Building Web Applications*, O'Reilly Media.
- [2] Kyle Simpson (2015), *You Don't Know JS (Book Series)*, O'Reilly Media.
- [3] Eric Evans (2003), *Domain-Driven Design: Tackling Complexity in the Heart of Software*, Addison-Wesley.
- [4] Martin Fowler (2002), *Patterns of Enterprise Application Architecture*, Addison-Wesley.
- [5] Robert C. Martin (2008), *Clean Code: A Handbook of Agile Software Craftsmanship*, Prentice Hall.